

《计算机科学导论》课程

数据库系统

上海交通大学计算机科学导论课程组



致谢：本课件由高远宁@CS2017D.SJTU编写，刘雨桐@CS2017D.SJTU修缮排版，孔令和添加内容，高晓沅终校定稿。

问题背景

新学期伊始，大学生需要在教务处系统上选择本学期课程
学生与课程信息繁杂，信息更新频繁



如何依据学生与课程数据库，快速查询所需数据？





设计理论

架构体系

发展历程

01. 设计理论：数据库简述

数据处理和管理是计算机应用最重要的领域之一，涉及社会生活的方方面面：

- **银行业**：所有交易
- **航空公司**：预订，时间表
- **大学**：注册，成绩
- **销售**：客户，产品，购买
- **制造业**：生产，库存，订单，供应链
- **电子政务，电子商务**，

学生成绩登记表

学号	姓名	课程	学分	成绩
3023001093	李家杰	数据库	4	92
3023001093	王伟	操作系统	5	88
3020621034	刘佳琪	英语	3	70
3020831035	吕松	微积分	2	85
3021131123	赵万里	数据库	4	95

 **数据库**：存放数据的仓库，用户可以进行新增、截取、更新、删除等操作

01. 设计理论：学校教务处系统

学校教务处系统需要管理多种数据

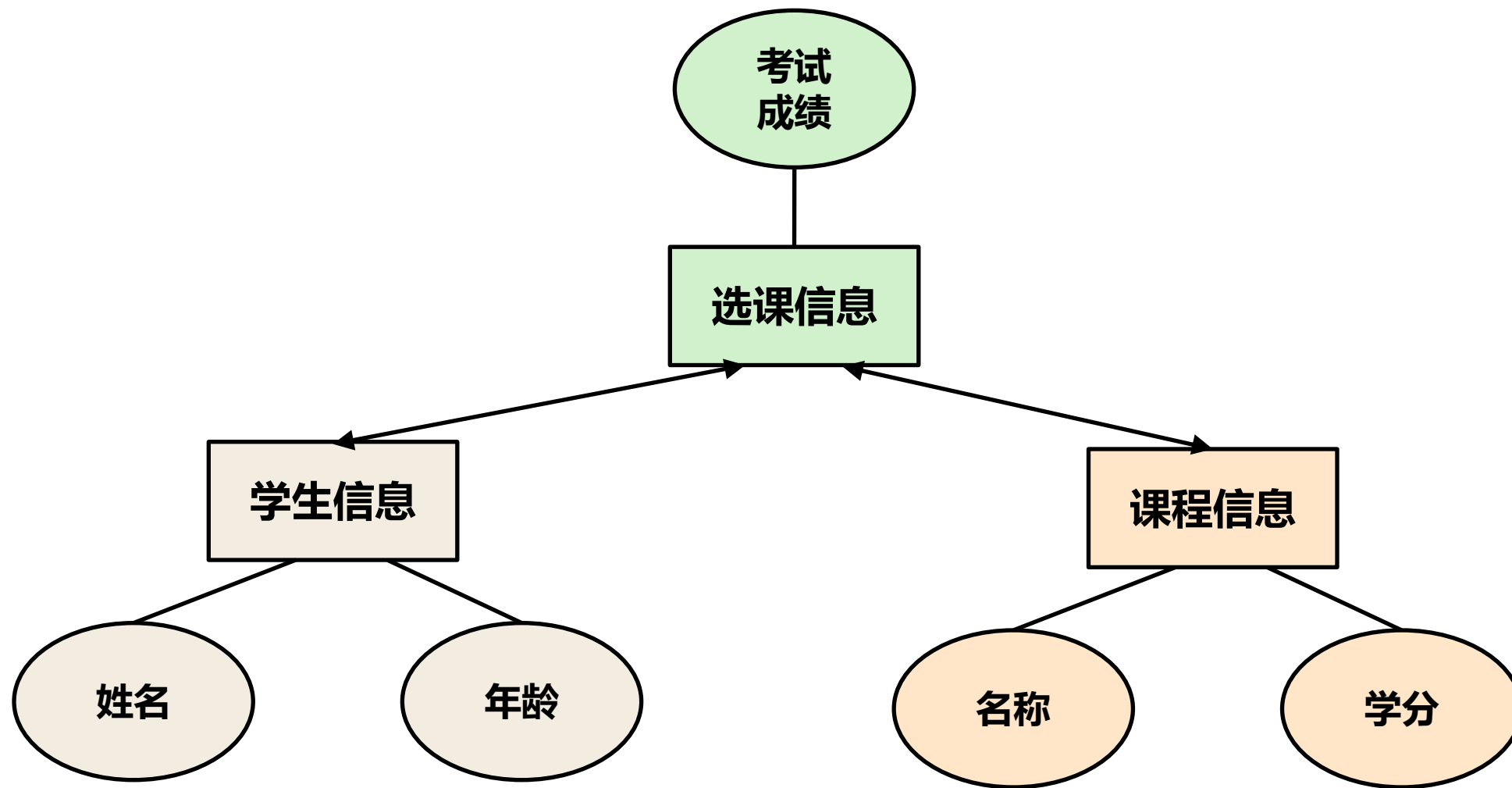
- 学生个人信息
- 课程表信息
- 学生选课信息

？ 如何根据数据之间的关系，
对它们进行有效存储

姓名	年龄	课程名称	学分	成绩
Sname	Sage	Cname	Credit	Grade
李家杰	21	数据库	4	92
王伟	20	操作系统	5	88
刘佳琪	18	英语	3	70
吕松	19	微积分	2	85
赵万里	22	数据库	4	95

学生课程信息

01. 设计理论：数据存储建模



学生信息与课程信息通过 **选课信息** 建立联系

01. 设计理论：关系型数据存储

关系型数据库存储目标

- 根据实体之间的关系，建立多个表格
- 多个表格分开存储，独立管理

姓名	年龄	课程名称	学分	成绩
Sname	Sage	Cname	Credit	Grade
李家杰	21	数据库	4	92
王伟	20	操作系统	5	88
刘佳琪	18	英语	3	70
吕松	19	微积分	2	85
赵万里	22	数据库	4	95

01. 设计理论：关系模型

关系： 由一个无序的集合组成，属性之间的关系构成实体

- 一个数据表代表了一个关系

属性： 关系中的特征集合

- 数据表中的每一列代表了一个属性

记录： 关系中的一行，是一系列属性值的集合

Student(Sname, Sage)

Sname	Sage
李家杰	21
王伟	20
刘佳琪	18
吕松	19
赵万里	22

一个N维关系
=
一个N列的表

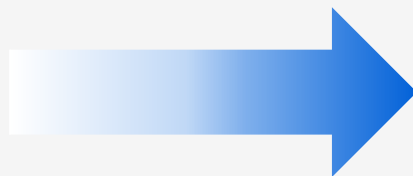
01. 设计理论：主键

主键： 在一个关系中唯一标识了一个元组，主键是递增的
数据库管理系统会自动帮助关系创建一个内部的主键

结构化查询语言 (SQL) : **Structured Query Language**

Student(Sname, Sage)

Sname	Sage
李家杰	21
王伟	20
刘佳琪	18
吕松	19
赵万里	22



Student(Sid, Sname, Sage)

Sid	Sname	Sage
3023001093	李家杰	21
3011112340	王伟	20
3020621034	刘佳琪	18
3020831035	吕松	19
3021131123	赵万里	22

01. 设计理论：外键

外键：用于与另一张表的关联

- 确定另一张表记录的字段
- 用于保持数据的一致性

Enrolled(Sid, Cid, Grade)

Sid	Cid	Grade
3023001093	1	92
3023001093	2	88
3020621034	1	70
3020831035	1	85
3021131123	2	95

Student(Sid, Sname, Sage)

Sid	Sname	Sage
3023001093	李家杰	21
3011112340	王伟	20
3020621034	刘佳琪	18
3020831035	吕松	19
3021131123	赵万里	22

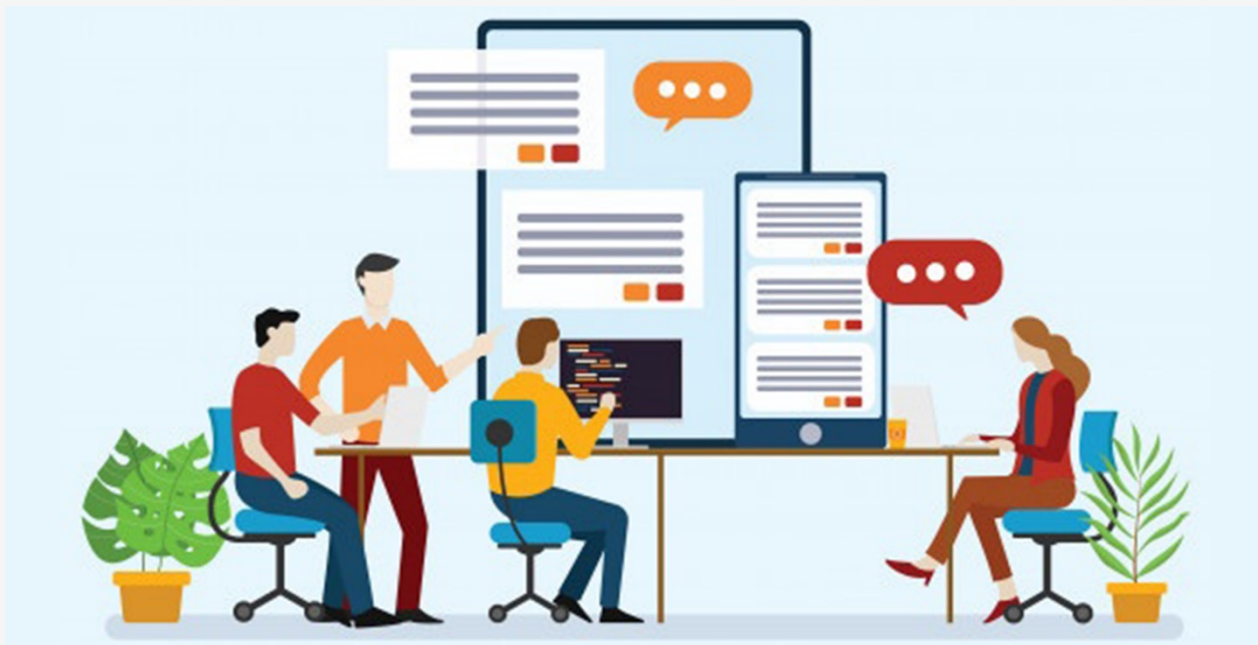
Course(Cid, Cname, Credit)

Cid	Cname	Credit
1	数据库	4
2	操作系统	5
3	英语	3
4	微积分	2

01. 设计理论：选课信息查询

为了统计信息，教务老师会进行多种查询，例如：

- 查询所有选择了**数据库**课程，年龄在**20岁以下**，成绩大于**90分**的学生姓名
- 查询所有**选择数学课程**，但**没有选物理课程**的学生学号和姓名



01. 设计理论：查询操作

关系代数：定义在数据表中查询与操控元组的基本操作

- 基于集合代数

每一个操作符输入一个或多个关系，输出一个新的关系

- 把多个操作符链接起来可以构成一个更加复杂的操作

σ 选择 (Select)

π 投影 (Projection)

$\cup$ 并 (Union)

$\cap$ 交 (Intersection)

$-$ 差 (Difference)

$\times$ 笛卡尔积 (Product)

01. 设计理论：选择

选择：该操作作用于从关系R中选取数据

- 结果被存储在一个结果表中（称为结果集）
- 运算符： σ
- 语法： $\sigma_{predicate}(R)$

例：查询选修了课程号为2和成绩大于90的选课记录

```
SELECT * FROM Enrolled
WHERE Cid= '2' AND Grade>90;
```

Enrolled(Sid, Cid, Grade)

Sid	Cid	Grade
3023001093	1	92
3023001093	2	88
3020621034	1	70
3020831035	1	85
3021131123	2	95



$\sigma_{Cid='2' \wedge Grade>90}(\text{Enrolled})$

Sid	Cid	Grade
3021131123	2	95

01. 设计理论：投影

投影：从关系R中选择出若干属性列组成新的关系

- 可以对属性值进行重新排序
- 可以修改属性值
- 运算符： π
- 语法： $\pi_{(A1,A2,...,An)}(R)$

例：在课程表中查询选修了课程号为2的学生学号和成绩

```
SELECT Sid,Grade, FROM Enrolled
WHERE Cid= '2' ;
```

Enrolled(Sid, Cid, Grade)

Sid	Cid	Grade
3023001093	1	92
3023001093	2	88
3020621034	1	70
3020831035	1	85
3021131123	2	95



$\pi_{\text{Sid,Grade}}(\sigma_{\text{Cid}='2'}(\text{Enrolled}))$

Sid	Grade
3023001093	88
3021131123	95

01. 设计理论：并/交

并： 由所有关系中包含的所有元组集合组成一个新的关系

- 语法： $(R \cup S)$

- 例：

```
(SELECT * FROM C1)
      UNION
      (SELECT * FROM C2);
```

交： 由在所有关系中均出现的元组集合组成一个新的关系

- 语法： $(R \cap S)$

- 例：

```
(SELECT * FROM C1)
      INTERSECT
      (SELECT * FROM C2);
```

C1(Cid, Cname, Credit)

Cid	Cname	Credit
1	数据库	4
2	操作系统	5
3	英语	3

C2(Cid, Cname, Credit)

Cid	Cname	Credit
2	操作系统	5
4	微积分	2
5	物理	3

$(C1 \cup C2)$

Cid	Cname	Credit
1	数据库	4
2	操作系统	5
3	英语	3
4	微积分	2
5	物理	3

去重Cid=2

$(C1 \cap C2)$

Cid	Cname	Credit
2	操作系统	5

01. 设计理论：差

差：由在第一个关系中出现不在第二个关系中出现的元组集合组成一个新的关系

- 语法：($R - S$)

例：查询在课程表C1中出现不在C2中出现的记录

```
(SELECT * FROM C1)
  EXCEPT
(SELECT * FROM C2);
```

C1(Cid, Cname, Credit) **C2**(Cid, Cname, Credit)

Cid	Cname	Credit
1	数据库	4
2	操作系统	5
3	英语	3

Cid	Cname	Credit
2	操作系统	5
4	微积分	2
5	物理	3



(C1 - C2)

Cid	Cname	Credit
1	数据库	4
3	英语	3

01. 设计理论：笛卡尔积

笛卡尔积：用来连接多个关系，构成一个更大的关系

- 运算符： $\times$
- 语法： $(R \times S)$

例：将C1与C2两个课程表进行记录与记录之间的连接

```
SELECT * FROM C1
CROSS JOIN C2;
```

```
SELECT * FROM C1,C2;
```

C1(Cid, Cname, Credit)

Cid	Cname	Credit
1	数据库	4
2	操作系统	5
3	英语	3

C1(Cid, Cname, Credit)

Cid	Cname	Credit
2	操作系统	5
4	微积分	2
5	物理	3

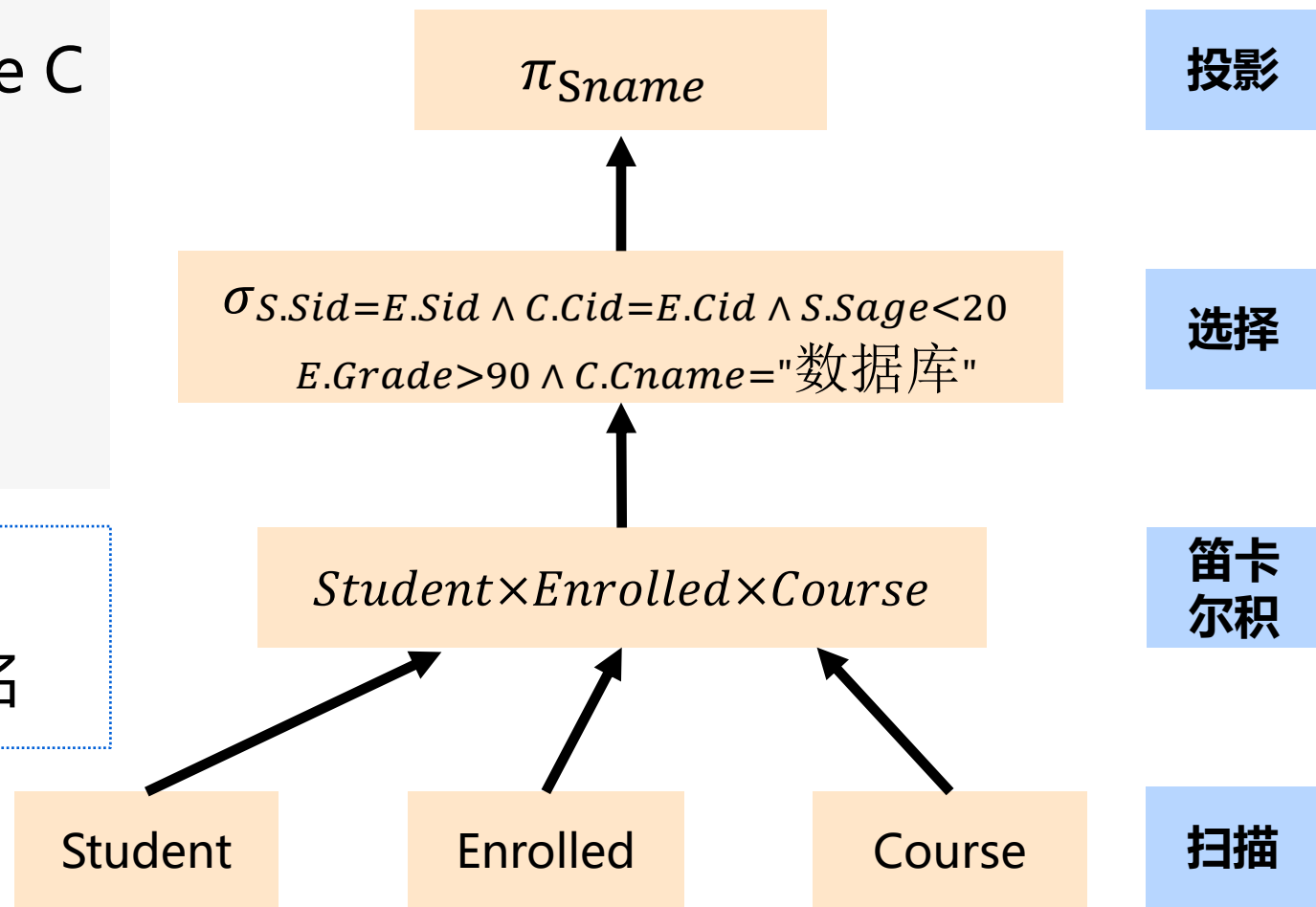


C1.Cid	C1.Cname	C1.Credit	C2.Cid	C2.Cname	C2.Credit
1	数据库	4	2	操作系统	5
1	数据库	4	4	微积分	2
1	数据库	4	5	物理	3
2	操作系统	5	2	操作系统	5
2	操作系统	5	4	微积分	2
2	操作系统	5	5	物理	3
3	英语	3	2	操作系统	5
3	英语	3	4	微积分	2
3	英语	3	5	物理	3

01. 设计理论：查询与关系代数图

```
SELECT Sname
FROM Student S, Enrolled E, Course C
WHERE S.Sid=E.Sid
      and C.Cid=E.Cid
      and C.Cname= "数据库"
      and S.Sage<20
      and E.Grade>90
```

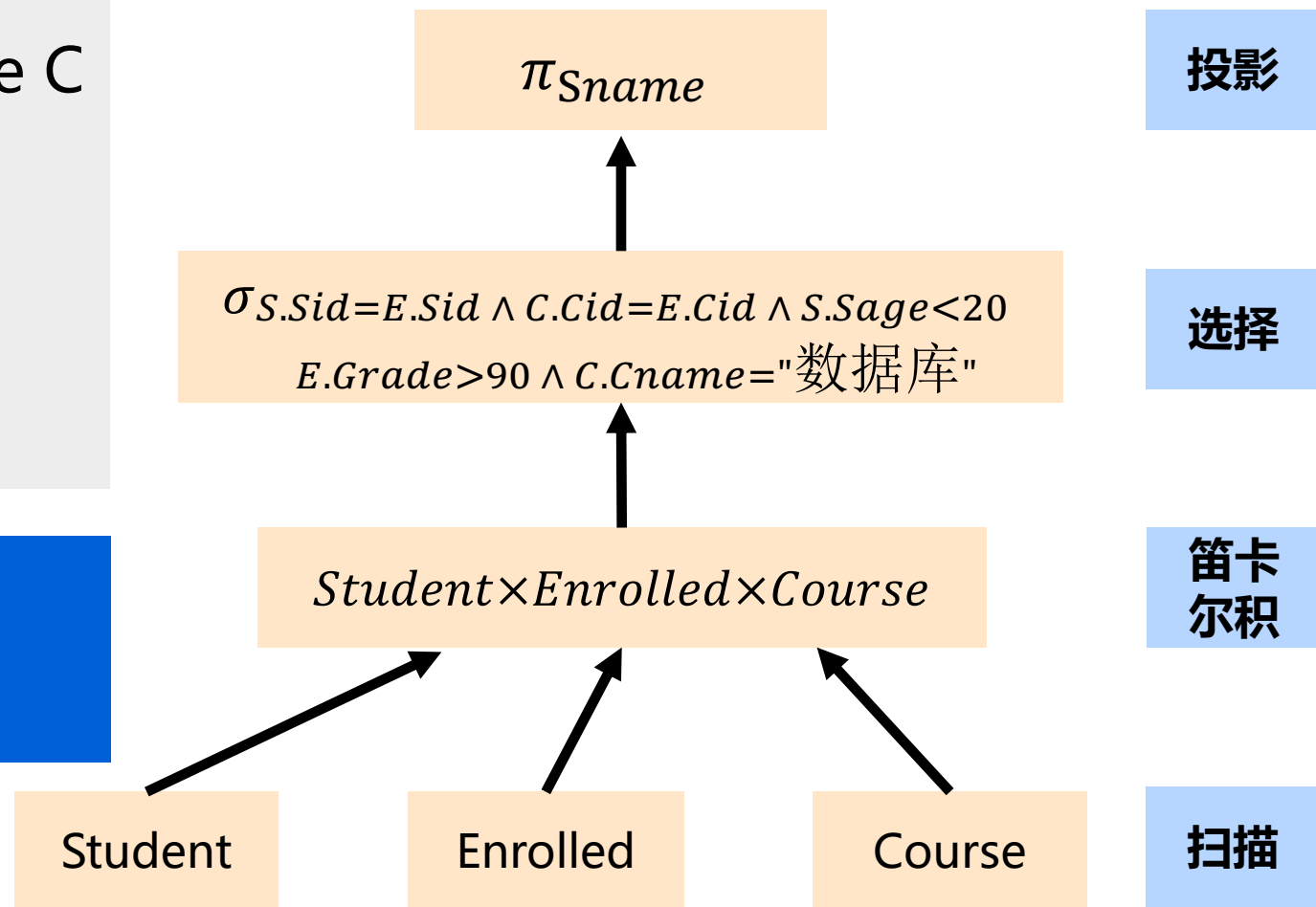
查询所有选择了**数据库**课程，年龄在**20岁以下**，成绩大于**90分**的学生姓名



01. 设计理论：查询与关系代数图

```
SELECT Sname
FROM Student S, Enrolled E, Course C
WHERE S.Sid=E.Sid
      and C.Cid=E.Cid
      and C.Cname= "数据库"
      and S.Sage<20
      and E.Grade>90
```

? 如何调整关系代数图的顺序，
从而优化数据库的查询过程



01. 设计理论：关系代数定律

1. 笛卡尔积结合率

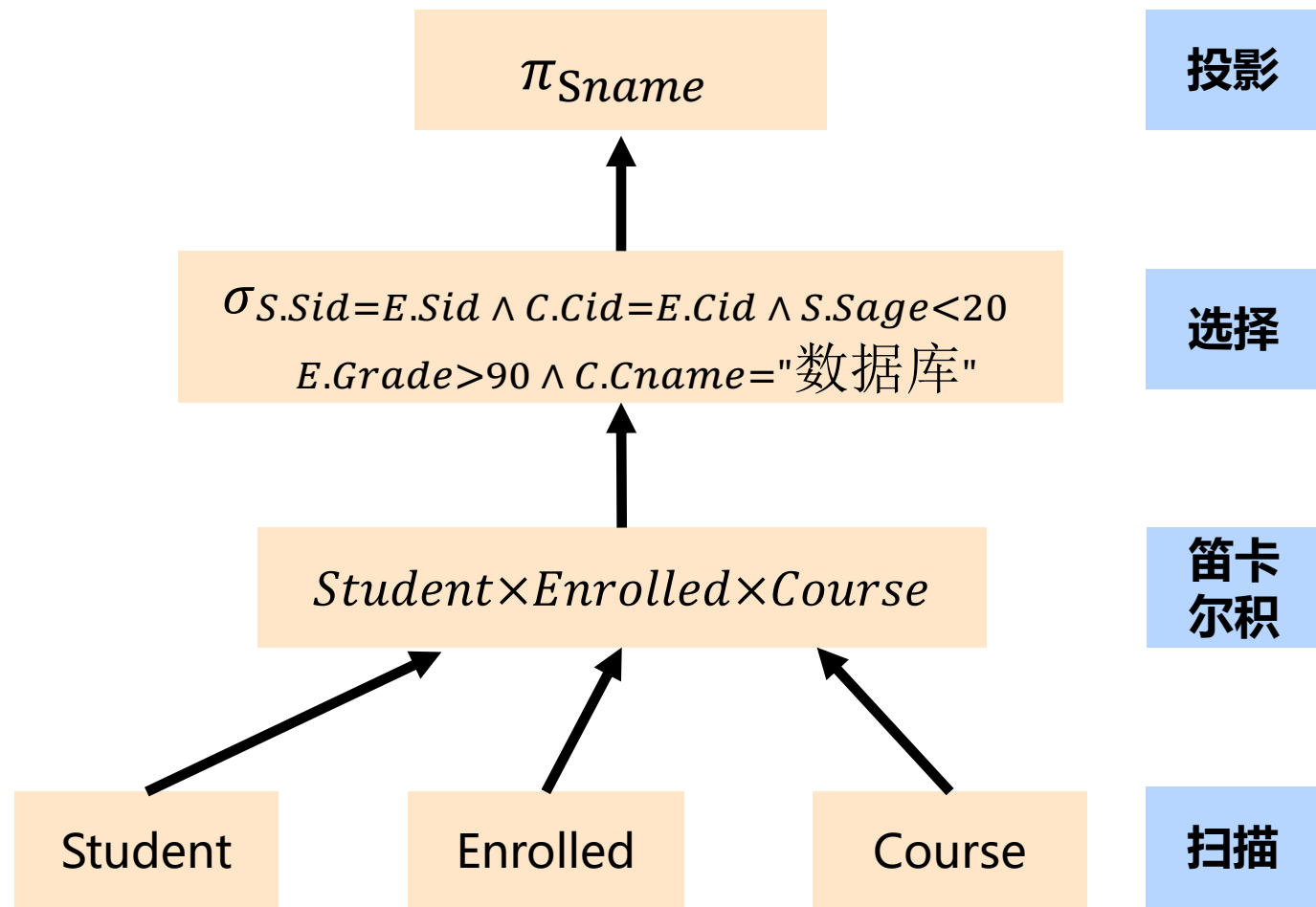
令 E_1, E_2, E_3 为三个关系, 则
 $(E_1 \times E_2) \times E_3 = E_1 \times (E_2 \times E_3)$

2. 选择的串联定律

令 F_1, F_2 为两个选择条件, 则
 $\sigma_{F_1 \wedge F_2}(E) = \sigma_{F_1}(\sigma_{F_2}(E))$

3. 选择与笛卡尔积的交换

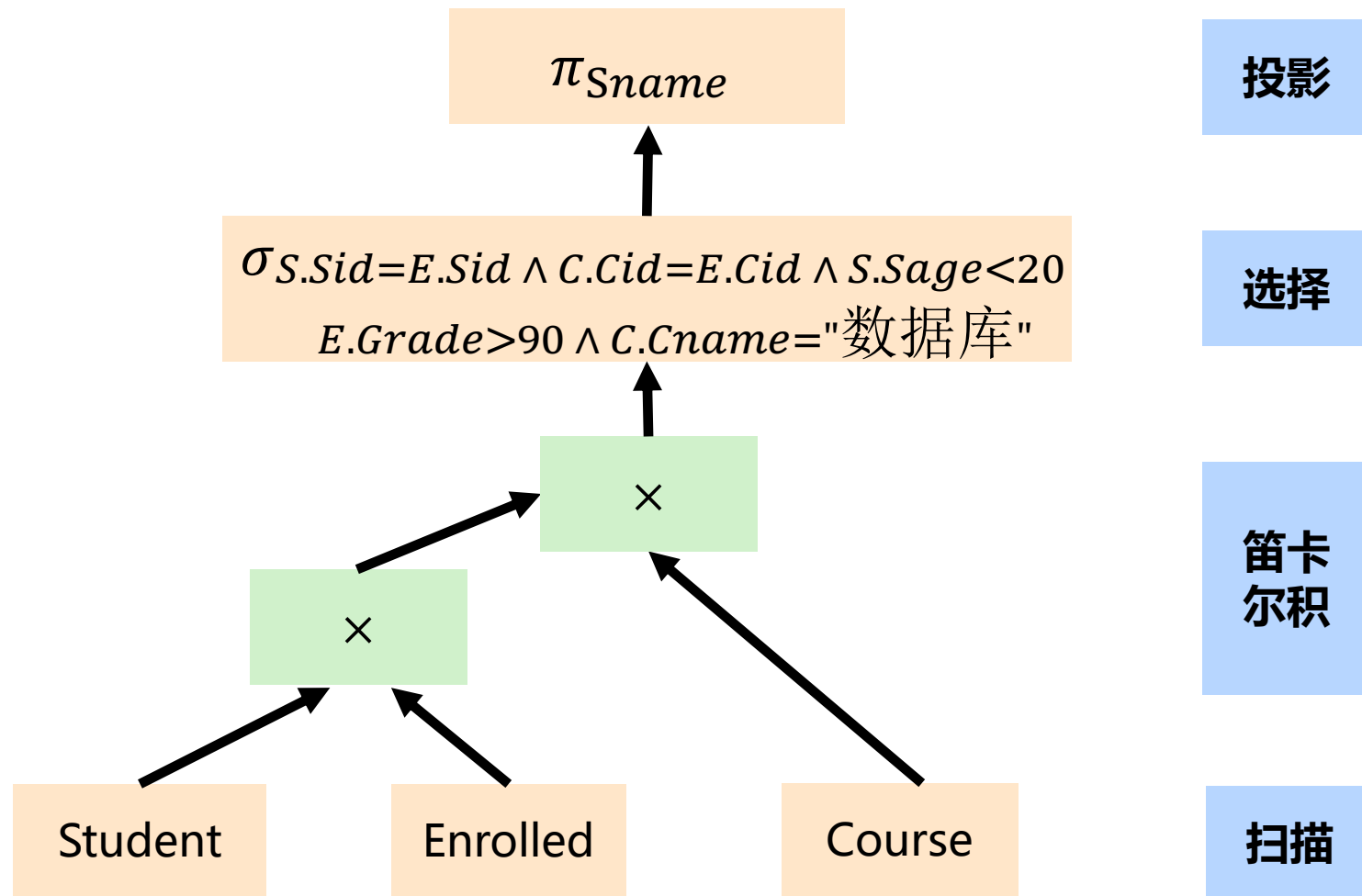
F_1 仅与 E_1 有关, F_2 仅与 E_2 有关, 则
 $\sigma_{F_1 \wedge F_2}(E_1 \times E_2) = \sigma_{F_1}(E_1) \times \sigma_{F_2}(E_2)$



01. 设计理论：关系代数图优化

1. 笛卡尔积结合

将笛卡尔积操作进行拆分，两两结合进行计算

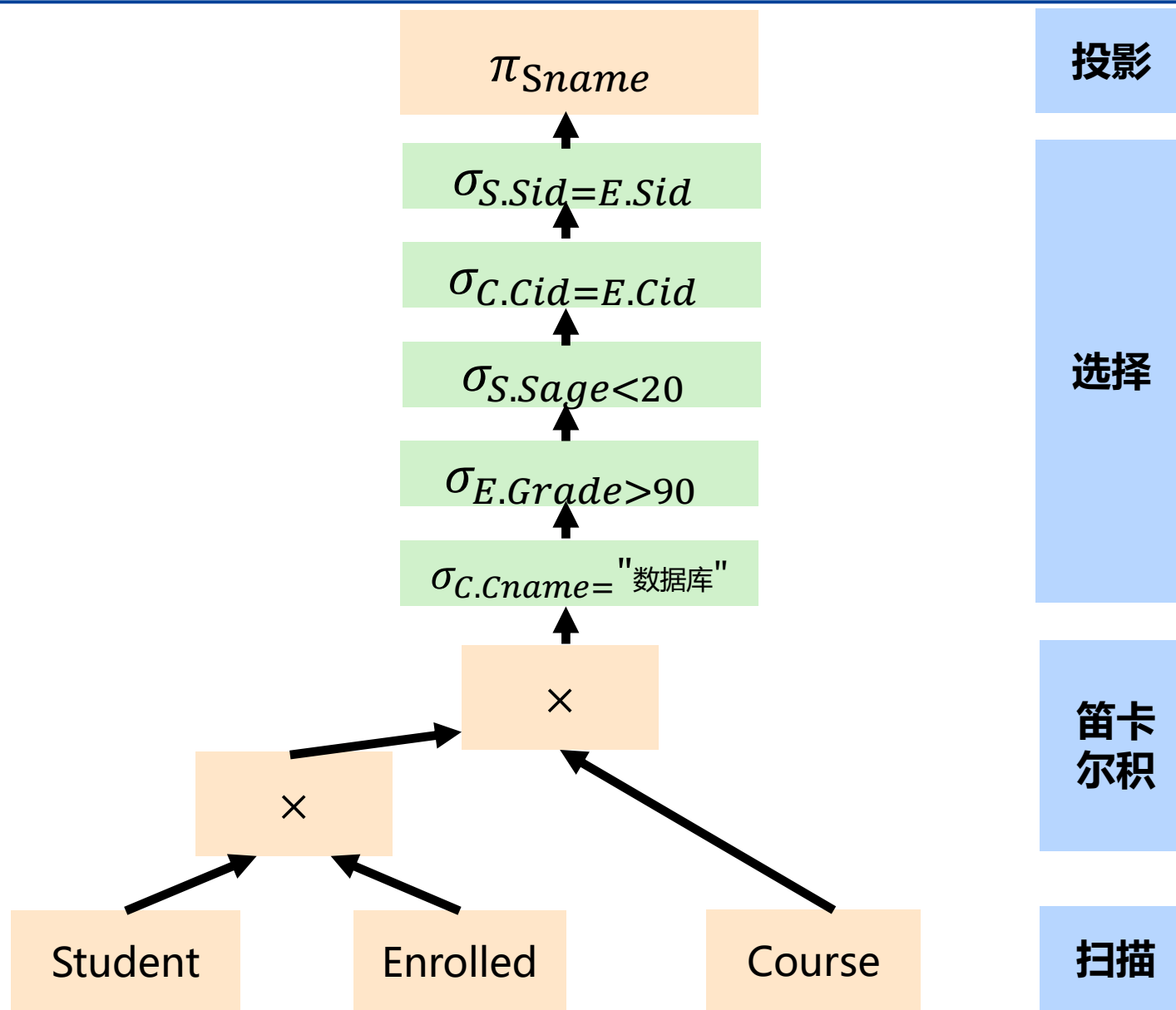


01. 设计理论：关系代数图优化

1. 笛卡尔积结合

2. 选择操作串联

选择操作进行拆分，依次计算



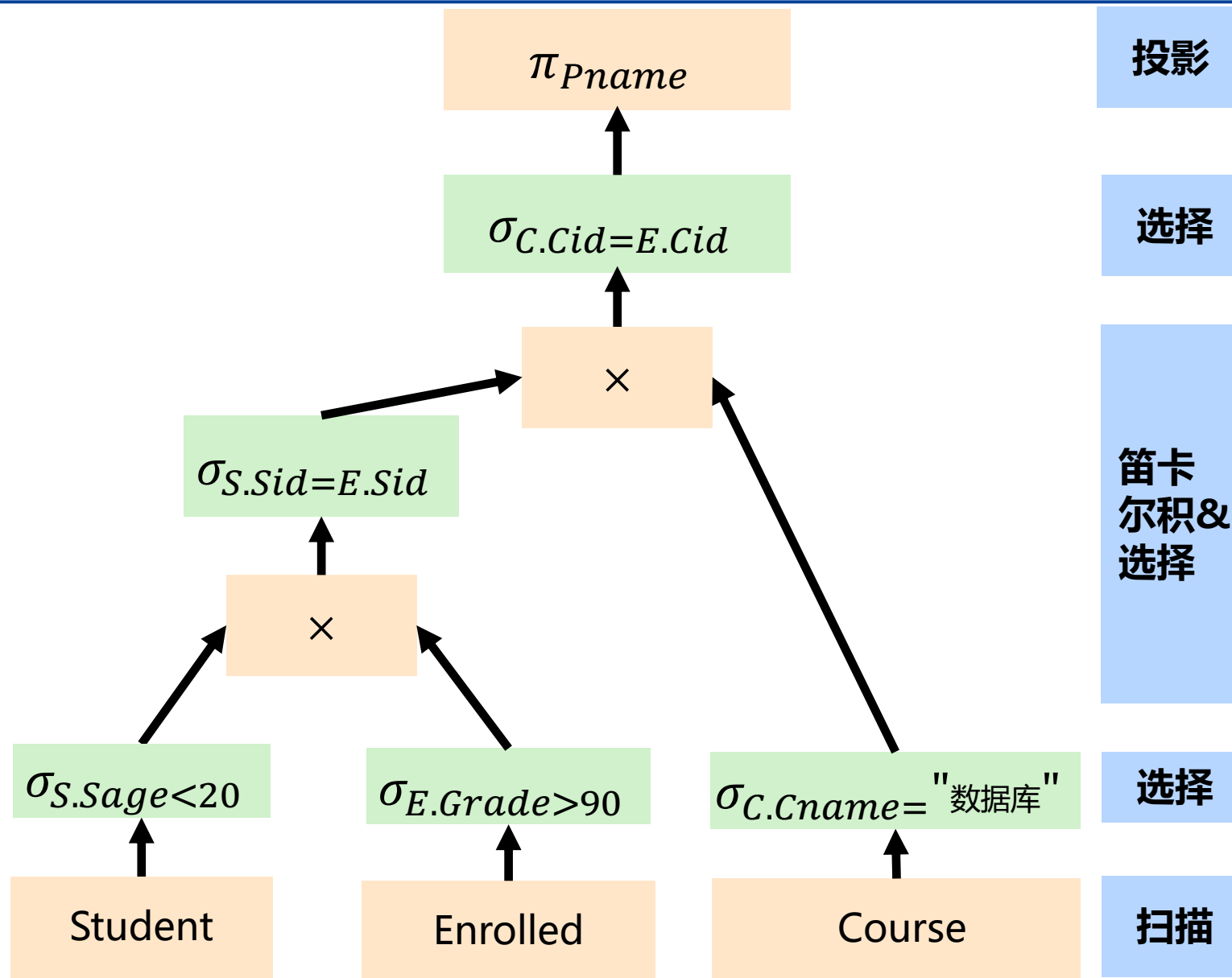
01. 设计理论：关系代数图优化

1.笛卡尔积结合

2.选择操作串联

3.选择与笛卡尔积交换

将选择操作下推，提前过滤无效数据



01. 设计理论：优化分析

分析中间过程的笛卡尔积大小

原始关系代数图

$$1\text{万} * 100 * 10\text{万} = \mathbf{1000}\text{亿}$$

最优关系代数图

$$(1\text{万} * 10\%) * (100 * 1\%) * (10\text{万} * 20\%) = \mathbf{0.2}\text{亿}$$

学生信息表

总共1万条记录
20岁以下记录占比10%

课程信息表

总共100条记录
数据库课程记录占比1%

选课信息表

总共10万条记录
90分以上记录占比20%

01. 设计理论：连接

连接：从两个关系中选取属性间满足一定条件的元组

- 语法：($R \bowtie S$)

例：查询在课程表C1和C2中出现的记录

```
SELECT * FROM C1  
NATURAL JOIN C2;
```

C1(Cid, Cname, Credit) **C2**(Cid, Cname, Credit)

Cid	Cname	Credit
1	数据库	4
2	操作系统	5
3	英语	3

Cid	Cname	Credit
2	操作系统	5
4	微积分	2
5	物理	3



($C1 \bowtie C2$)

Cid	Cname	Credit
2	操作系统	5

01. 设计理论：更多查询示例

查询未选择任何课程的学生学号

- $\pi_{Sid}(Student) - \pi_{Sid}(Enrolled)$
- `SELECT Sid FROM Student MINUS SELECT Sid FROM Enrolled`

查询选修了编号为 “2” 的课程的学生姓名和其年龄

- $\pi_{Sname, Sage}(\sigma_{Cid="2"}(Student \bowtie Enrolled))$
- `SELECT Sname, Sage FROM Students NATURAL JOIN Enrolled
WHERE Cid= "2"`

查询选修了课程名为 “Math” 的学生学号和姓名

- $\pi_{Sid, Sname}(\sigma_{Cname="Math"}(Student \bowtie Enrolled \bowtie Course))$
- `SELECT Sid, Sname FROM Students
NATURAL JOIN Enrolled NATURAL JOIN Course
WHERE Cname = "Math"`



设计理论

架构体系

发展历程

02. 架构体系：数据

数据(Data)是数据库中存储的基本对象

定义

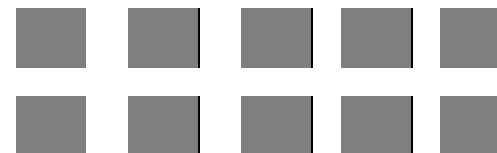
- 描述事物的符号记录

种类

- 文本、图形、图像、音频、视频、学生的档案记录、货物的运输情况等

特点

- 数据与其语义是不可分的



02. 架构体系：数据库

定义

- 数据库(Database,简称DB)是长期储存在计算机内、有组织的、可共享的大量数据的集合。

组织

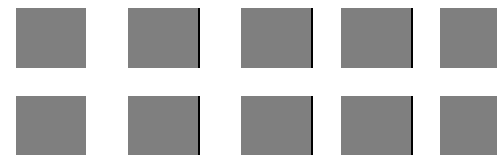
- 数据按一定的数据模型组织，数据相互关联。

基本特征

- 数据按一定的数据模型组织、描述和储存
- 可为各种用户共享
- 冗余度较小
- 数据独立性较高
- 易扩展



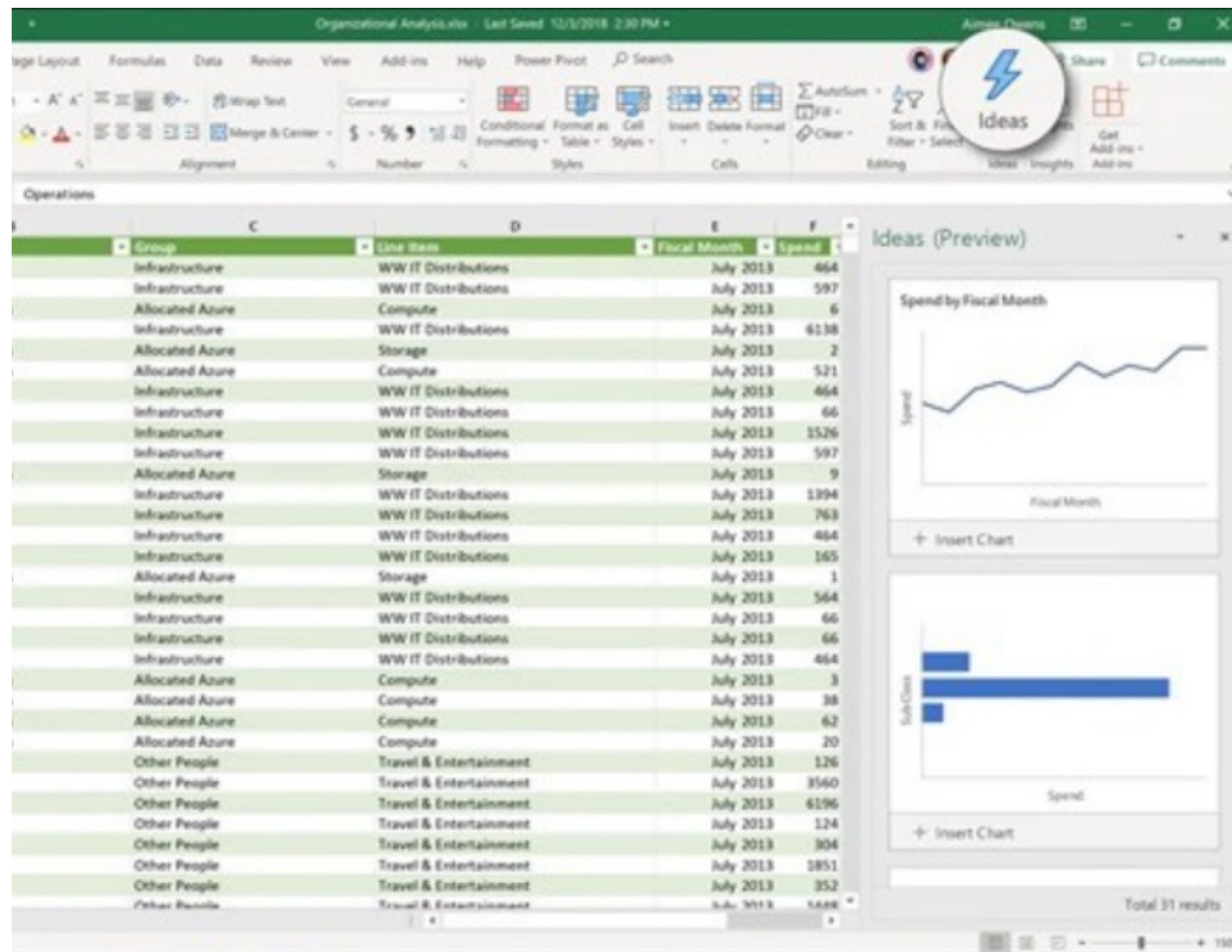
存储数据



02. 架构体系：数据库功能

- 数据库包含两层意思
 - 数据库是一个实体，它是能够合理保管数据的“仓库”，数据和库组成了数据库。
 - 数据库是数据管理的新方法和技术，能更合适的组织数据、更方便的维护数据、更严密的控制数据和更有效的利用数据。

【问题】Excel电子表格是数据库吗？



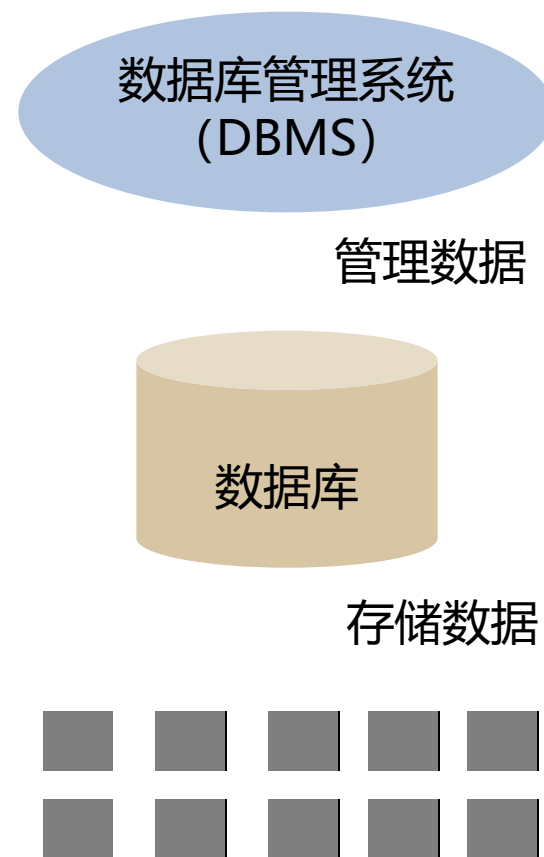
02. 架构体系：数据库管理系统（DBMS）

概念

- 用于建立、使用和维护数据库的系统软件。
位于用户与操作系统之间的数据管理软件。

特点

- 专用于实现对数据进行管理和维护的系统软件
- 利用DBMS能科学地组织和存储数据、高效地获取和维护数据
- 如：SQL Server, Oracle, Sybase, Mysql



02. 架构体系：数据库应用程序与用户使用

数据库应用程序

- 运用数据库管理系统所支持的程序设计语言编写
- 如：Visual Basic, Delphi, Java, Python

功能：创建并处理表单等

- 处理用户查询
- 创建并处理报表
- 执行应用逻辑
- 控制应用

数据库应用程序

...

数据库应用程序

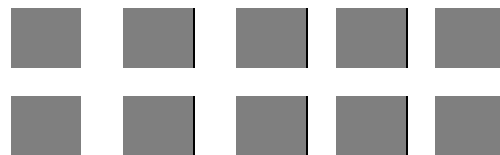
使用数据

数据库管理系统
(DBMS)

管理数据

数据库

存储数据



02. 架构体系：数据库分类

- 关系型数据库

- 关系型数据库，存储的格式可以直观地反映实体间的关系。
- 关系型数据库和常见的表格比较相似，关系型数据库中表与表之间是有很多复杂的关联关系的。常见的关系型数据库有Mysql，SqlServer等。

- 非关系型数据库（NoSQL）

- 随着近些年技术方向的不断拓展，大量的NoSql数据库如MongoDB、Redis、Memcache出于简化数据库结构、避免冗余、影响性能的表连接、摒弃复杂分布式的目的被设计。
- NoSQL数据库适合追求速度和可扩展性、业务多变的应用场景。存储方式包括键值对、图结构或者文档等。

02. 架构体系：关系型与非关系型数据库区别

数据库类型	特性	优点	缺点
关系型数据库 SQLite、Oracle、mysql	1、关系型数据库，是指采用了关系模型来组织数据的数据库； 2、关系型数据库的最大特点就是事务的一致性； 3、简单来说，关系模型指的就是二维表格模型，而一个关系型数据库就是由二维表及其之间的联系所组成的一个数据组织。	1、容易理解：二维表结构是非常贴近逻辑世界一个概念，关系模型相对网状、层次等其他模型来说更容易理解； 2、使用方便：通用的SQL语言使得操作关系型数据库非常方便； 3、易于维护：丰富的完整性(实体完整性、参照完整性和用户定义的完整性)大大减低了数据冗余和数据不一致的概率； 4、支持SQL，可用于复杂的查询。	1、为了维护一致性所付出的巨大代价就是其读写性能比较差； 2、固定的表结构； 3、高并发读写需求； 4、海量数据的高效率读写；
非关系型数据库 MongoDb、redis、HBase	1、使用键值对存储数据； 2、分布式； 3、一般不支持ACID特性； 4、非关系型数据库严格上不是一种数据库，应该是一种数据结构化存储方法的集合。	1、无需经过sql层的解析，读写性能很高； 2、基于键值对，数据没有耦合性，容易扩展； 3、存储数据的格式：nosql的存储格式是key,value形式、文档形式、图片形式等等，文档形式、图片形式等等，而关系型数据库则只支持基础类型。	1、不提供sql支持，学习和使用成本较高； 2、无事务处理，附加功能b和报表等支持也不好；

Not only SQL





设计理论

架构体系

发展历程

03. 发展历程：数据库系统的发展

1960s

集成数据存储

美国Honeywell公司的IDS (Integrated Data Store) 系统投入运行，揭开了数据库技术的序幕。



1980s

关系数据库

关系系统由于使用简便以及硬件性能的改善，逐步代替网状系统和层次系统占领了市场。

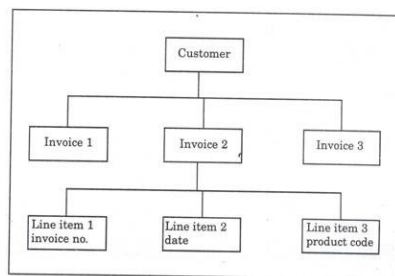
ORACLE®



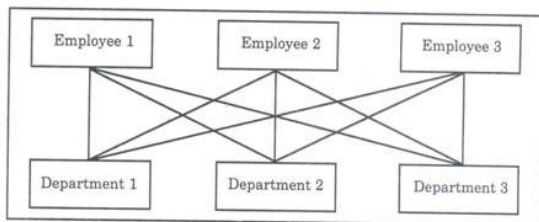
1970s

关系模型

数据库蓬勃发展的年代，**层次模型**、**网状模型**占据了整个数据库商用市场，首次提出关系模型。



层次模型



网状模型

Relational Model

Activity Code	Activity Name
23	Patching
24	Overlay
25	Crack Sealing

Key = 24

Activity Code	Date	Route No.
24	01/12/01	1-95
24	02/08/01	1-66

Date	Activity Code	Route No.
01/12/01	24	1-95
01/15/01	23	1-495
02/08/01	24	1-66

关系模型

2000s

数据仓库

大数据时代到来，数据仓库，NoSQL 逐渐涌现。

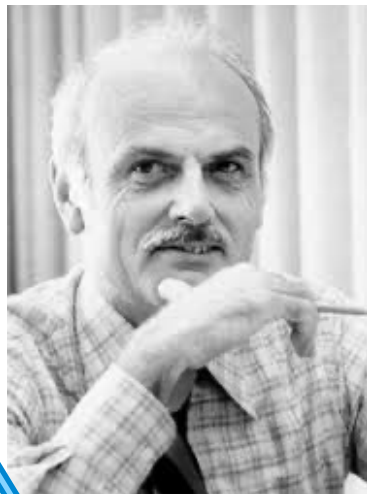
03. 发展历程：数据库领域杰出人士

图灵奖
得主



1973

查理士·巴赫曼
(Charles Bachman)
首创网状模型理论



1981

埃德加·科德
(E. F. Codd)
首创关系模型理论



1998

詹姆斯·格雷
(Jim Gray)
革新事务处理技术



2014

迈克尔·斯通布雷克
(Michael Stonebraker)
现代数据库系统底层开发



拉里·埃里森
(Larry Ellison)

- 读了三个大学，没有一个学位文凭
- 换了十多家公司
- 创业资金只有1200美元

发现关系型
数据库商机



- 建立甲骨文 (Oracle)，成为世界第二大软件公司
- 成为硅谷首富

03. 发展历程：数据库系统使用排名 (来源: <https://db-engines.com/en/ranking>)

- Complete ranking
- Relational DBMS
- Key-value stores
- Document stores
- Time Series DBMS
- Graph DBMS
- Search engines
- Vector DBMS
- Object oriented DBMS
- RDF stores
- Wide column stores
- Multivalue DBMS
- Spatial DBMS
- Native XML DBMS
- Event Stores
- Content stores
- Navigational DBMS
- Columnar
- Time Series, Observability
- NoSQL

427 systems in ranking, December 2025

Rank			DBMS	Database Model	Score		
Dec 2025	Nov 2025	Dec 2024			Dec 2025	Nov 2025	Dec 2024
1.	1.	1.	Oracle	Relational, Multi-model ⓘ	1234.40	-5.38	-29.39
2.	2.	2.	MySQL	Relational, Multi-model ⓘ	868.49	+2.67	-135.27
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model ⓘ	722.52	+3.65	-83.18
4.	4.	4.	PostgreSQL	Relational, Multi-model ⓘ	659.42	+8.06	-6.95
5.	5.	5.	MongoDB ⚡	Multi-model ⓘ	372.27	+0.60	-28.11
6.	6.	↑7.	Snowflake	Relational	202.45	+4.61	+55.09
7.	7.	↓6.	Redis	Key-value, Multi-model ⓘ	142.48	-2.61	-7.79
8.	8.	↑13.	Databricks	Multi-model ⓘ	138.02	+6.51	+50.33
9.	9.	9.	IBM Db2	Relational, Multi-model ⓘ	115.75	-3.52	-7.03
10.	10.	↓8.	Elasticsearch	Multi-model ⓘ	109.83	-4.14	-22.49
11.	11.	↓10.	SQLite	Relational	102.61	-1.58	+0.89
12.	12.	↓11.	Apache Cassandra	Wide column, Multi-model ⓘ	101.55	-1.16	+3.62
13.	13.	↑15.	MariaDB ⚡	Relational, Multi-model ⓘ	87.59	+0.23	+3.82
14.	↑16.	14.	Splunk	Search engine	75.89	-0.21	-9.47
15.	↑18.	↑18.	Apache Hive	Relational	75.44	+2.57	+22.34
16.	↓15.	16.	Microsoft Azure SQL Database	Relational, Multi-model ⓘ	75.22	-1.16	-1.15
17.	↓14.	↓12.	Microsoft Access	Relational	73.89	-4.40	-16.93
18.	↓17.	↓17.	Amazon DynamoDB	Multi-model ⓘ	72.96	-2.82	+0.23
19.	19.	19.	Google BigQuery	Relational	62.76	-0.03	+10.46
20.	20.	20.	Neo4j	Graph	51.76	-0.60	+8.69

03. 发展历程：数据库系统市场

关系数据库管理系统的公司：

- 甲骨文（Oracle）、SAP（Sybase）：最大的数据库软件公司之一
- IBM（DB2）：世界上最大的DBMS供应商之一
- 微软的SQL-Server以及Access：精简、相对便宜

关系数据库公司的挑战

- 随着互联网web2.0网站的兴起，为了解决大规模数据集合多重数据种类带来的挑战，NoSQL数据库应运而生

其他数据库产品：Ingres, Paradox, Foxbase, FoxPro, dBase,...

ORACLE®

 **SYBASE®**

 Microsoft®
SQL Server®

03. 发展历程：中国数据库企业

产品	公司	背景
OceanBase	蚂蚁金服	自研
Polar DB	阿里云	基于MySQL开发
GaussDB	华为	基于PostgreSQL 开发
TDSQL	腾讯	基于 MySQL 开发
TiDB	PingCAP	基于 Spanner 论文自研
SequoiaDB	巨杉	自研



关系数据库系统的设计理论

- 数据查询
- 关系代数

数据库系统的架构体系

- 数据、数据库
- 数据库管理系统
- 数据库应用程序
- 关系型与非关系型数据库

数据库系统的发展历程

- 背景
- 产生与发展
- 我国数据库产业



缩写	会议全称
SIGMOD	ACM Conference on Management of Data
SIGKDD	ACM Knowledge Discovery and Data Mining
VLDB	International Conference on Very Large Data Bases
ICDE	IEEE International Conference on Data Engineering
TKDE	IEEE Transactions on Knowledge and Data Engineering

谢谢!

